Unified Detection of Machine-Generated Code Across Languages, Generators, and Usage Scenarios

Divyanshu Giri[1] **Nikhita Ravi**[1] **Shrish Kadam**[1]
[1]International Institute of Information Technology, Hyderabad
{divyanshu.giri, nikhita.ravi, shrish.kadam}@research.iiit.ac.in

Abstract

The rise of generative models has made it increasingly difficult to distinguish machine-generated code from human-written code especially across different programming languages, domains, and generation techniques. his project aims to address SemEval-2026 Task 13, which focuses on building systems that can detect machine-generated code under diverse conditions by evaluating generalization to unseen languages, generator families, and code application scenarios.
We address three subtasks: binary detection of machine-generated code (Subtask A), multi-class authorship attribution across LLM families (Subtask B), and hybrid code detection (Subtask C). Building on the provided CodeBERT baseline, we correct several implementation issues and improve performance using class-weighted loss, label smoothing, stratified subsampling to mitigate class imbalance, and cosine learning rate scheduling. Our best CodeBERT model achieves a Macro F1 of 0.4203 on Subtask B after 5 epochs of stable training with no overfitting, while a UniXcoder-based model reaches a higher Macro F1 of 0.470 but exhibits clear overfitting (validation loss rising 22% after epoch 3), making the CodeBERT result the more reliable of the two.

1 Introduction

Distinguishing between human-written and machine-generated code is a critical challenge in software engineering. SemEval-2026 Task 13 defines this problem through three subtasks: Subtask A (Binary Detection), Subtask B (Multi-Class Authorship), and Subtask C (Hybrid Code Detection). The baseline model provided by the organizers utilizes C ODE BERT (Feng et al., 2020), a bimodal pre-trained model for programming and natural languages. While effective, the baseline treats code primarily as sequence data, potentially missing structural and data-flow nuances that differentiate human logic from generative patterns.

SemEval-2026 Task 13 (**?**) formalizes this problem through three subtasks of increasing complexity:

- **Subtask A** (Binary Detection): Classify code as *human-written* or *machine-generated*.
- **Subtask B** (Multi-Class Authorship): Identify the author as human or one of 10 LLM families.
- **Subtask C** (Hybrid Detection): Distinguish fully human, fully machine, hybrid, and adversarial code.

In this phase, we primarily address the first two subtasks. For Subtask A, we experiment with backbone models (CodeBERT, UniXcoder) and classification head architectures (Bi-LSTM, dense blocks with stylometric features). For Subtask B, we identify and fix critical bugs in the baseline weight loss implementation, then systematically address the extreme class imbalance through sqrt-scaled class-weighted loss, label smoothing, stratified subsampling, and cosine scheduling. We conduct rigorous hyperparameter search over learning rate and sequence length, achieving substantial improvements over the baseline. For Subtask C, we have reproduced the official CodeBERT baseline for 4-class hybrid code detection (human, machine, hybrid, adversarial) and established reference validation performance. Our next step would be to improve upon this code by using GraphCodeBERT leveraging structural and data-flow nuances that differentiate human logic from generative patterns. We were constrained by the compute restrictions and were not able to directly use it now.[1]

[1]https://github.com/DivyG007/procastic-simulaters-SemEval-2026-Task13

2 Background and Related Work

2.1 Code-Oriented Pre-trained Models

Several pre-trained transformer-based models which were contained inside the constraints of this task are:

- **CodeBERT** (**?**): A bimodal model pre-trained on natural language–programming language pairs using masked language modeling and replaced token detection. It serves as the official task baseline.
- **GraphCodeBERT** (**?**): Extends CodeBERT by incorporating *data flow graphs* (DFGs) during pre-training, enabling the model to capture semantic-level code structure such as variable dependencies and data flow paths.
- **UniXcoder** (**?**): A unified cross-modal pre-trained model that leverages AST (Abstract Syntax Tree) representations alongside code tokens.
- **StarCoder / StarEncoder** (**?**): Trained on The Stack, a large multilingual code corpus, providing strong cross-lingual code representations.

2.2 Some Other Relevant Benchmarks on Machine-Generated Code Detection

The DROID resource suite (**?**) is the first large-scale benchmark specifically designed for AI-generated code detection, providing code samples across multiple languages, generators, and domains. The CoDet-M4 system (**?**) established initial baselines using CodeBERT fine-tuning, demonstrating that while transformer-based models achieve strong in-distribution performance, generalization to unseen languages and domains remains a significant challenge.

The CodeXGLUE benchmark (**?**) provides a comprehensive suite of code understanding and generation tasks, informing our choice of evaluation methodology and pre-trained model selection.

3 Task Description

We now describe the three subtasks we address in detail.

3.1 Subtask A: Binary Machine-Generated Code Detection

Given a code snippet, the goal is to classify it as either *fully human-written* or *fully machine-generated*.

- **Training data**: 500K samples (238K human / 262K machine) in C++, Python, and Java from the algorithmic domain (e.g., competitive programming).
- **Validation data**: 100K samples.
- **Evaluation**: Performance is measured across four generalization settings (Table **??**).

Setting	Language	Domain
(i) Seen L, Seen D	C++, Py, Java	Algorithmic
(ii) Unseen L, Seen D	Go, PHP, C#, etc.	Algorithmic
(iii) Seen L, Unseen D	C++, Py, Java	Research, Production
(iv) Unseen L, Unseen D	Go, PHP, C#, etc.	Research, Production

Table 1: Subtask A evaluation matrix: Models are tested across seen/unseen language and domain combinations.

3.2 Subtask B: Multi-Class Authorship Detection

Given a code snippet, predict its author: human, or one of 10 LLM families—DeepSeek-AI, Qwen, 01-ai, BigCode, Gemma, Phi, Meta-LLaMA, IBM-Granite, Mistral, and OpenAI.

- **Training data**: 500K samples with severe class imbalance—442K human-written ($\sim$88%) vs. 2K–10K per LLM family.
- **Validation data**: 100K samples.
- **Generalization**: Evaluation includes "unseen authors"—specific models not present during training but belonging to known LLM families.

The extreme class imbalance (Table **??**) makes Subtask B particularly challenging, as naive models tend to predict the majority class (human) for most samples.

3.3 Subtask C: Hybrid Code Detection

Given a code snippet, the goal is to classify it into one of four categories reflecting how the code was produced:

Class	Count	%
Human	442,000	88.4
OpenAI	10,000	2.0
Meta-LLaMA	8,000	1.6
Qwen	8,000	1.6
IBM-Granite	8,000	1.6
Phi	5,000	1.0
DeepSeek-AI	4,000	0.8
Mistral	4,000	0.8
01-ai	3,000	0.6
BigCode	2,000	0.4
Gemma	2,000	0.4

Table 2: Subtask B training set class distribution showing severe imbalance.

- **Human-written** (label 0): Code written entirely by a human developer.
- **Machine-generated** (label 1): Code generated entirely by an LLM.
- **Hybrid** (label 2): Code that was partially written by a human and partially completed or edited by an LLM (or vice versa).
- **Adversarial** (label 3): Code generated via adversarial prompting or RLHF techniques specifically designed to mimic human-written code and evade detection.
- **Training data**: 900K samples with moderate class imbalance—485K human-written ($\sim$54%), 210K machine-generated ($\sim$23%), 118K adversarial ($\sim$13%), and 85K hybrid ($\sim$9%).
- **Validation data**: 200K samples.
- **Evaluation**: Macro F1-score, consistent with Subtasks A and B.

Subtask C is uniquely challenging because it requires distinguishing not only human from machine code, but also *degrees of machine involvement*. The hybrid class contains code with mixed authorship, and the adversarial class contains code intentionally crafted to deceive detectors—both of which are expected to share surface-level features with fully human-written code. Table **??** summarizes the training set distribution.

4 Datasets

All data is provided by the task organizers via Kaggle and HuggingFace in Apache Parquet format. Each sample contains the following fields:

Class	Count	%
Human-written	485,000	53.9
Machine-generated	210,000	23.3
Adversarial	118,000	13.1
Hybrid	85,000	9.4

Table 3: Subtask C training set class distribution. While less extreme than Subtask B, the hybrid class is notably underrepresented.

- `code`: The source code snippet (string).
- `label`: Integer label ID mapped via `label_to_id.json`.
- `language`: Programming language metadata.
- `generator`: The name/family of the LLM that produced the code (or "human").

Label mappings (`label_to_id.json` and `id_to_label.json`) are provided in each task folder. We use the official training and validation splits without any additional external data, in compliance with the task rules.

Data Characteristics. We investigated the dataset while approaching the task and found several important properties:

1. The training set covers only three programming languages (C++, Python, Java) and the algorithmic domain, while evaluation extends to Go, PHP, C#, C, JavaScript, and the research/production domains.
2. Subtask B suffers from extreme class imbalance, with the human class comprising approximately 88% of training samples.
3. Subtask C has a more balanced but still skewed distribution (54% human, 23% machine, 13% adversarial, 9% hybrid). The hybrid and adversarial classes are particularly challenging because they share surface-level features with both fully human and fully machine code.

5 Proposed Methodology

We reorganized our approach into five phases, building incrementally from the baseline towards an iteratively improved system with some phases blocked out due to compute constraints.

5.1 Phase 1: Baseline Reproduction and Analysis

We first reproduce the official CodeBERT baseline (**?**) using the provided `baselines/train.py` script. This establishes reference Macro F1 scores on the validation set for Subtasks A, B, and C, and provides a foundation for error analysis.

The baseline uses `microsoft/codebert-base` (125M parameters) with a linear classification head, trained with standard cross-entropy loss and the AdamW optimizer. Input code is tokenized using the RoBERTa tokenizer with truncation at 512 tokens.

For Subtask C, the same CodeBERT architecture is used with `num_labels=4` (human, machine, hybrid, adversarial). The baseline trains on a 10% subsample of the 900K training set ($\sim$90K samples) and evaluates on a corresponding subsample of the 200K validation set. This establishes the reference performance before applying the class-imbalance and architecture improvements developed for Subtasks A and B.

5.2 Phase 2: Classification Head Engineering

The baseline uses a single linear layer on top of the `[CLS]` token embedding. We experiment with more expressive classification on Subtask-B:

- **Bi-LSTM Head**: A bidirectional LSTM over the full sequence of token embeddings, followed by attention pooling. This captures sequential dependencies in the feature space that may distinguish human from machine patterns.
- **Dense Blocks**: Multi-layer perceptron with batch normalization, ReLU activation, and high dropout ($p = 0.3$–0.5) to prevent overfitting on the training languages and domain.
- **Auxiliary Feature Concatenation**: Optionally concatenating metadata features (programming language embedding, code length, comment density) with the transformer output before classification.

5.3 Phase 3: Handling Class Imbalance (Subtask B)

The severe class imbalance in Subtask B (88% human) is the single largest issue to achieving drastically improved Macro F1 scores. Naive training with standard cross-entropy loss causes the model to predict the majority class (human) for nearly all inputs, achieving $\sim$88% accuracy but near-zero F1 on all 10 LLM classes. We implement and evaluate the following strategies:

- **Sqrt-Scaled Class-Weighted Loss**: We compute inverse-frequency class weights with square-root dampening (Equation **??**) and pass them to `torch.nn.CrossEntropyLoss` via a custom `WeightedTrainer` overriding `Trainer.compute_loss()`. The scaling prevents over-correction on rare classes (e.g., BigCode and Gemma with $\sim$2K samples).

$$w_c = \sqrt{\frac{N}{C \cdot n_c}} \tag{1}$$

  where N is the total (subsampled) training size, $C = 11$ the number of classes, and n_c the count of class c.

- **Label Smoothing**: Hard one-hot targets are replaced with smoothed targets ($\alpha = 0.1$), reducing overconfidence on the dominant human class and improving calibration:

$$y'_c = (1 - \alpha)\, y_c + \frac{\alpha}{C} \tag{2}$$

- **Stratified Subsampling**: The human class is capped at `HUMAN_SUBSET_SIZE` (20K–40K) while all minority LLM classes are retained. This reduces the dataset from 500K to $\sim$78K–98K samples, yielding a less skewed distribution while preserving minority data.
- **Cosine Learning Rate Schedule**: Cosine annealing with linear warmup (6% of steps) ensures smooth decay, avoiding unstable early updates and premature convergence.
- **Gradient Accumulation**: Batch size 16 with 2 accumulation steps gives an effective batch size of 32, balancing GPU memory limits (16 GB Kaggle T4) with gradient stability.
- **Early Stopping**: Training monitors validation Macro F1 with patience of 3 evaluation rounds to prevent overfitting.

Planned approaches. We also considered hierarchical classification (first binary human-vs-machine, then LLM family identification) and weighted random sampling via PyTorch's `WeightedRandomSampler`. These remain promising directions for future work but were not implemented in the current system due to the effectiveness of the class-weighted loss approach.

5.4 Phase 4: Ensemble Learning

To maximize robustness, especially for the challenging unseen-language and unseen-domain evaluation settings, we combine predictions from multiple diverse models:

- CODEBERT (**?**)
- GRAPHCODEBERT (**?**)
- UNIXCODER (**?**)

Ensemble strategies include:

1. **Soft Voting**: Averaging predicted probability distributions across models.
2. **Weighted Averaging**: Assigning higher weights to models that perform better on the validation set.
3. **Stacking**: Training a meta-learner (logistic regression) on the concatenated prediction vectors of base models.

Model diversity is key: CodeBERT captures token-level patterns, GraphCodeBERT captures data-flow structure, and UniXcoder leverages AST information. The ensemble pipeline will be applied to all three subtasks, with task-specific weight tuning.

5.5 Phase 5: Structure-Aware Modeling with GraphCodeBERT (Future Implementation

We hypothesize that machine-generated code exhibits different data-flow properties compared to human-written code. Human developers tend to write code with more diverse variable naming, irregular control flow, and context-dependent patterns, while LLMs often produce code with more uniform structure. To exploit these differences, we replace the CodeBERT backbone with **GraphCodeBERT** (**?**), which incorporates data flow graph (DFG) information during pre-training. This allows the model to capture:

- **Variable dependency patterns**: How variables flow through computations to potentially distinct between human and machine code.
- **Long-range semantic relationships**: DFG edges connect distant tokens that are semantically related, providing richer features than sequential attention alone.
- **Language-agnostic structure**: Data flow patterns are more abstract than surface syntax, potentially improving generalization to unseen languages.

Due to compute constraints we refrained from implementing this immediately despite this seeming better in theory and developed upon dataset and resource constraints right now.

6 Evaluation Metrics

The primary evaluation metric for all subtasks is **Macro F1-score** directed as per the task leaderboard, which computes the unweighted mean of per-class F1 scores. In addition to Macro F1, we report:

- **Accuracy**: Overall correctness.
- **Per-class Precision, Recall, and F1**: For detailed error analysis across classes.
- **Setting-wise breakdown** (Subtask A): Separate scores for seen/unseen language $\times$ seen/unseen domain to assess generalization.

7 Experimental Results

7.1 Baseline Results

Task	Model	Macro F1	Accuracy
A	CodeBERT (baseline)	0.34	0.33
B	CodeBERT (baseline, buggy)	$\sim$0.22*	0.88
C	CodeBERT (baseline)	0.51	0.81

*Estimated Macro F1 after fixing the `average='weighted'` metric bug (see Section **??**). Accuracy is high because the model predicts the majority class (human) for nearly all samples.

Table 4: Baseline results on the validation set.

7.2 Subtask A: Generalization Analysis

7.2.1 Methodology: Classification Heads

Three classification head configurations placed upon backbone models **CodeBERT** and **UniXcoder** are explored:

- **Linear head**: A single linear projection from the `[CLS]` pooled representation to the output logits.

- **Dense head**: Multiple fully connected layers with non-linear activations and dropout regularization, applied on top of the pooled `[CLS]` representation.

- **BiLSTM head**: The full token-level sequence output from the transformer encoder is passed through a bidirectional LSTM, and the final hidden states from both directions are concatenated to form the input to the classification layer. This allows the model to capture long-range sequential dependencies beyond what mean/CLS pooling provides.

7.2.2 Methodology: Stylometric Feature Inclusion and Smoothing

Stylometric features encode surface-level and structural patterns in code that reflect authorial style independent of semantic content. Seven hand-crafted stylometric features are extracted per sample in the ablation experiments such as average line length, code complexity, ratio of comment lines to total lines and variable name length.

7.2.3 Experimental Setup

Due to computational constraints, we ran ablation studies on a smaller subset of the data (2,000 training samples and 500 each for validation and testing). This allowed us to quickly evaluate the contribution of different features, identify useful components, and discard ineffective ones.

Training configuration. All six ablation models share the following settings: AdamW optimiser, base learning rate 2×10^{-5}, per-device batch size 16, maximum sequence length 512 tokens, up to 10 epochs, and random seed 42. The *pure backbone* variants (rows 1–2) use standard cross-entropy loss, weight decay 0.01, and a linear LR schedule with no warmup. The *BiLSTM* variants (rows 3–4) use a cosine LR schedule with 10% linear warmup and early stopping (patience = 3); the BiLSTM head has hidden size 256 (bidirectional $\rightarrow$ 512-dim), a single dense block (LayerNorm + GELU), and dropout 0.15. The *stylometric fusion* variants (rows 5–6) apply label smoothing ($\alpha = 0.05$), weight decay 0.02, gradient accumulation over 2 steps (effective batch 32), and a differential learning rate scheme: backbone receives 2×10^{-5} while the newly initialised head layers receive 2×10^{-4} ($10\times$ multiplier). Early stopping patience is 5.

Here are the results:

Model	Class 0 (Human)		Class 1 (AI-Gen.)		Accuracy	Macro F1	Wtd. F1	Δ Macro F1
	P	F1	P	F1				
CodeBERT-Base (Baseline)	0.83	0.47	0.25	0.37	0.42	0.42	0.45	—
UniXcoder-Base	0.91	0.23	0.24	0.38	0.31	0.30	0.26	−0.12 ↓
CodeBERT + BiLSTM + Dense	0.88	0.32	0.25	0.39	0.35	0.35	0.34	−0.07 ↓
UniXcoder + BiLSTM + Dense	0.80	0.23	0.23	0.36	0.30	0.29	0.26	−0.13 ↓
CodeBERT + Smoothing + 7 Sty.	0.82	0.47	0.24	0.37	0.42	0.42	0.45	≈ 0.00
UniXcoder + Smoothing + 7 Sty.†	0.92	0.39	0.26	0.41	0.40	0.40	0.39	−0.02 ↓

7.2.4 Discussion

UniXcoder degrades performance. **UniXcoder** performs worse than CodeBERT, suggesting that structural and data-flow features are less useful than surface-level cues for distinguishing AI from human code.

BiLSTM Heads Underperform with Limited Data Both BiLSTM variants underperform simpler heads, likely because the extra parameters add complexity without benefit given the small (2,000 sample) training set.

UniXcoder without tuning underperforms CodeBERT. Naively swapping the backbone drops macro F1 from 0.42 to 0.30. UniXcoder's richer pre-training makes it more sensitive to fine-tuning conditions; without label smoothing or adapted hyperparameters, its structural priors misfire on this task distribution.

7.2.5 Final Model

Based on the ablation findings, our final Subtask A model is **CodeBERT** fine-tuned with tuned hyperparameters, early stopping, 8 stylometric features, and 10,000 training samples.

Model	Accuracy	Macro F1
CodeBERT (Baseline)	0.40	0.39
CodeBERT + Stylometry + Tuning (Final)	0.40	0.40

Table 5: Subtask A final model vs. baseline on the validation set.

7.2.6 Future Work

Full-dataset evaluation. The most immediate next step is re-running the ablation on the full training set. The BiLSTM regression and the near-perfect UniXcoder result are plausibly data-scale artifacts that warrant rigorous re-evaluation under the complete class distribution.

GraphCodeBERT with DFG generation pipeline. GraphCodeBERT (**?**) incorporates data flow graphs (DFGs) as structural inputs. We plan to implement a DFG extraction pipeline using tree-sitter and fine-tune GraphCodeBERT with

these signals, since AI-generated code may exhibit subtly different data flow patterns—such as redundant variable chains—making DFG-augmented representations a theoretically motivated extension.

Ensemble learning over best-performing models. The complementary failure modes of CodeBERT and UniXcoder variants suggest ensembles could yield more robust classifiers. We plan to explore soft-voting and stacking over the best configurations, potentially combined with a lightweight gradient-boosted model trained on stylometric features as a diversity-maximizing third member.

Cross-language and cross-generator generalization. Stratified evaluation per programming language and per generator will clarify whether models learn genuine authorship signals or exploit language-level distributional artifacts that would not generalize across corpora.

7.3 Subtask B: Multi-Class Authorship Detection

7.3.1 Baseline Diagnosis and Bug Analysis

Before developing improved models, we conducted a thorough understadning of the codebase of the provided Subtask B baseline code. This revealed **six implementation bugs**(some critical and some pretty trivial) that collectively rendered the baseline results misleading and significantly degraded model performance. Table **??** catalogues each bug, its root cause, and its impact.

ID	Bug Description and Impact
B1	**Wrong metric.** Used `average='weighted'` instead of `'macro'` F1, inflating scores by favoring the 88% majority class.
B2	**Tensor corruption.** `return_tensors='pt'` in `map()` caused silent data corruption during batching.
B3	**Double padding.** Redundant `padding=True` with `DataCollatorWithPadding`.
B4	**No class weighting.** Standard CE loss with 88.4% human skew $\Rightarrow$ model predicts majority class, Macro F1 ≈ 0.09.
B5	**No early stopping.** Fixed epochs without val. loss monitoring led to overfitting.
B6	**Variable mismatch.** Prediction cell used `t` instead of `trainer_obj`.

Table 6: Critical bugs in the Subtask B baseline. All fixed prior to experimentation.

After fixing all six bugs, the corrected baseline CodeBERT model—still using standard cross-entropy loss with no class weighting—achieved Macro F1 ≈ 0.22 on the validation set, consistent with the leaderboard metric.

7.3.2 Training Improvements

Building on the corrected baseline, we implemented the following improvements (described in detail in Section **??**):

1. **Sqrt-scaled class-weighted CrossEntropyLoss** to directly address class imbalance in the loss function.
2. **Label smoothing** ($\alpha = 0.1$) to prevent overconfidence on the human class and improve calibration.
3. **Stratified subsampling** (cap human class at 20K–40K, keep all minority classes) to reduce training-set skew.
4. **Cosine LR schedule** with 6% warmup for smooth convergence.
5. **Gradient accumulation** (2 steps, effective batch size 32) for GPU memory efficiency.
6. **Early stopping** (patience = 3) to prevent overfitting.
7. **Mixed-precision training** (`fp16=True`) for faster iteration on Kaggle T4 GPU.

All experiments were conducted on a single NVIDIA Tesla T4 GPU (16 GB VRAM) via Kaggle Notebooks.

7.3.3 Experiment 1: UniXcoder with Initial Improvements

Our first improved experiment used `microsoft/unixcoder-base` (125.9M parameters) as the backbone, chosen for its AST-aware pretraining which we hypothesized would capture structural authorship patterns. Key settings: LR $= 2\times10^{-5}$, max_length $= 512$, 10 epochs, `HUMAN_SUBSET_SIZE` $=$ 20K, weight_decay $= 0.01$, no label smoothing, sqrt-scaled class weights, cosine schedule with 6% warmup, effective batch size 32. Table **??** shows the validation Macro F1 progression.

Analysis. The UniXcoder model reached a peak Macro F1 of **0.470** at step 6488 (epoch 5.72), a $>2\times$ improvement over the corrected baseline (0.22). However, two concerning patterns emerged:

1. *Validation loss divergence*: After reaching a minimum of 0.367 at epoch 3, the validation loss steadily increased to 0.449 by epoch 5.7—a 22% increase—indicating clear overfitting.

Epoch	Train Loss	Val. Loss	Macro F1
1.0	1.812	0.482	0.332
2.0	0.621	0.374	0.385
3.0	0.402	0.367	0.432
4.0	0.289	0.388	0.442
5.0	0.212	0.418	0.449
5.7	0.185	0.449	**0.470**

Table 7: Experiment 1 (UniXcoder) training dynamics. Val. loss rises after epoch 3 (overfitting), while Macro F1 peaks at epoch 5.7.

The model began memorizing training-set patterns rather than learning generalizable authorship features.

2. *Premature termination*: Training was forced to stop at epoch 5.72/10 due to disk quota exhaustion on the Kaggle environment (`Errno 28: No space left on device`), caused by excessive checkpoint storage without `save_total_limit`.

These observations motivated three changes for subsequent experiments: (a) reducing the maximum epoch count from 10 to 5, (b) increasing `HUMAN_SUBSET_SIZE` from 20K to 40K to provide more diverse human-class examples and reduce overfitting, and (c) adding label smoothing ($\alpha = 0.1$) and increasing weight decay (from 0.01 to 0.05) as additional regularizers.

7.3.4 Experiment 2: CodeBERT Hyperparameter Search

Based on the insights from Experiment 1, we conducted a systematic hyperparameter search using `microsoft/codebert-base` (124.7M parameters). We reverted to CodeBERT for two reasons: (1) UniXcoder's AST-aware pretraining did not yield clear advantages over CodeBERT for authorship attribution in preliminary comparisons, and (2) CodeBERT is the established competition baseline, enabling cleaner ablation. Key settings: `HUMAN_SUBSET_SIZE` = 40K, weight_decay = 0.05, label_smoothing = 0.1, sqrt-scaled class weights, cosine schedule, effective batch size 32, early stopping (patience = 3).

The 3×2 grid search (3 learning rates $\times$ 2 sequence lengths) evaluated six hyperparameter combinations, each for 2 epochs. Results are presented in Table **??**.

Key Observations.

Trial	LR	Max Len	Macro F1	Time
1	1×10^{-5}	128	0.2929	54 min
2	1×10^{-5}	256	0.3085	1h 38m
3	2×10^{-5}	128	0.3364	54 min
4	2×10^{-5}	256	**0.3676**	1h 38m
5	5×10^{-5}	128	0.3545*	—
6	5×10^{-5}	256	—†	—

*Completed ∼1.4/2 epochs before timeout. †Not completed due to session limits.

Table 8: HP search results for CodeBERT on Subtask B. Best: LR = 2×10^{-5}, max_length = 256.

1. **Learning rate sensitivity**: LR=2×10^{-5} consistently outperformed LR=1×10^{-5} across both sequence lengths (by 4–6 points), suggesting the lower rate is insufficient for the model to learn discriminative minority-class features within 2 epochs. The highest rate (5×10^{-5}) showed competitive partial performance (0.3545 at epoch 1.4), suggesting potential benefit from aggressive early learning, though it risks instability over longer training.

2. **Sequence length matters**: Increasing max length from 128 to 256 consistently improved Macro F1 by 1.6–3.1 percentage points, indicating that code authorship features are distributed throughout the snippet and cannot be fully captured in the first 128 tokens alone.

3. **Efficiency–performance trade-off**: Training with 256 tokens requires approximately $1.8\times$ the wall-clock time of 128-token models. The best configuration (Trial 4) achieves a Macro F1 of 0.3676 in 1 hour 38 minutes of training.

7.3.5 Experiment 3: Full Training with Best Hyperparameters

Using the best configuration identified in Experiment 2 (LR = 2×10^{-5}, max_length = 256), we trained CodeBERT for 5 full epochs with the same regularisation suite (label smoothing $\alpha = 0.1$, weight_decay = 0.05, sqrt-scaled class weights, cosine schedule, `HUMAN_SUBSET_SIZE` = 40K). Table **??** shows the training progression.

Analysis. CodeBERT reached Macro F1 = $\mathbf{0.4203}$ at epoch 5, a $+14\%$ absolute improvement over the 2-epoch HP search (0.368) and within 5 points of UniXcoder's peak (0.470). Crucially, unlike Experiment 1, the validation loss *decreases* monotonically (1.551→1.513), indicating that label smoothing ($\alpha = 0.1$) and higher weight decay (0.05) effectively prevent overfitting. The model

Epoch	Train Loss	Val. Loss	Macro F1
1	3.544	1.551	0.360
2	3.247	1.578	0.368
3	3.029	1.525	0.400
4	2.875	1.514	0.417
5	2.799	1.513	**0.420**

Table 9: Experiment 3 (CodeBERT, 5 epochs) training dynamics. Val. loss decreases throughout, unlike Experiment 1, confirming that the additional regularisation prevents overfitting.

still had not fully converged at epoch 5, suggesting further gains with extended training.

7.3.6 Per-Class Analysis of Best CodeBERT Model

Table **??** presents the per-class Precision, Recall, and F1 scores for the best CodeBERT model (Experiment 3: 5 epochs, LR=2×10^{-5}, max_length=256).

Class	Prec.	Rec.	F1
Human	1.00	0.90	0.95
DeepSeek-AI	0.12	0.46	0.20
Qwen	0.26	0.32	0.29
01-ai	0.18	0.34	0.24
BigCode	0.40	0.62	0.48
Gemma	0.20	0.61	0.30
Phi	0.50	0.51	0.50
Meta-LLaMA	0.28	0.37	0.32
IBM-Granite	0.48	0.73	0.58
Mistral	0.14	0.26	0.19
OpenAI	0.45	0.83	0.59
Macro Avg.	**0.36**	**0.54**	**0.42**

Table 10: Subtask B per-class results (Experiment 3: CodeBERT, LR=2×10^{-5}, max_length=256, 5 epochs, Macro F1 = 0.4203).

Class-level analysis. The per-class results reveal a clear stratification of LLM family detectability:

- **Easily detected** (F1 $\geq$ 0.45): OpenAI (0.59), IBM-Granite (0.58), Phi (0.50), and BigCode (0.48). These families likely produce code with distinctive stylistic signatures—e.g., characteristic comment patterns, variable naming conventions, or structural idioms that differ from human code and other LLM families.
- **Moderately detected** (F1 0.24–0.35): Meta-LLaMA (0.32), Gemma (0.30), Qwen (0.29), and 01-ai (0.24). These families are partially distinguishable but frequently confused with each other or with human code.
- **Poorly detected** (F1 $\leq$ 0.20): DeepSeek-AI (0.20) and Mistral (0.19). These are the hardest classes. Their low precision (Mistral: 0.14, DeepSeek: 0.12) indicates false-positive contamination, likely because their code generation style closely mimics human patterns or overlaps heavily with other LLM families.

Precision–Recall imbalance. IBM-Granite exhibits a notably asymmetric profile: high recall (0.73) but lower precision (0.48), meaning the model over-predicts this class—many samples from other classes are incorrectly labeled as IBM-Granite. This may indicate that IBM-Granite's code style serves as a "catch-all" pattern for ambiguous LLM-generated samples. Conversely, the human class shows near-perfect precision (1.00) but lower recall (0.90), meaning 10% of human-written code is misclassified as machine-generated.

7.4 Comparison Across Experiments

Table **??** summarizes the progression of Macro F1 across all Subtask B configurations.

Configuration	Epochs	Macro F1	Δ vs. Base
CodeBERT baseline (buggy)	3	$\sim$0.22	—
CodeBERT baseline (corrected)	3	$\sim$0.22	$\sim$0.00
UniXcoder + cls. wt. + sub-20K	5.7	0.470	+0.25
CodeBERT + cls. wt. + sub-40K + LS (HP search)	2	0.368	+0.15
CodeBERT + cls. wt. + sub-40K + LS (full)	5	**0.420**	+0.20

Table 11: Subtask B experimental progression. "cls. wt." = sqrt-scaled class weights, "sub-NK" = stratified subsample with human capped at NK, "LS" = label smoothing ($\alpha = 0.1$).

Comparing Experiments. Experiment 3 confirms that *training duration* is the dominant factor: CodeBERT's Macro F1 rose from 0.368 (2 epochs) to 0.420 (5 epochs), narrowing the gap with UniXcoder (0.470 at 5.7 epochs). At comparable epoch counts (epoch 2), UniXcoder achieved 0.385 vs. CodeBERT's 0.368, indicating only a modest backbone advantage. Crucially, Experiment 3's validation loss *decreased* throughout training (1.551$\rightarrow$1.513), whereas Experiment 1's rose by 22% (0.367$\rightarrow$0.449), demonstrating that label smoothing and higher weight decay effectively prevent overfitting and leave room for further gains

with extended training.

7.5 Ablation: Impact of Individual Components

To understand the contribution of each improvement, Table **??** provides a qualitative assessment based on our experimental observations.

Component	Impact
Bug fixes (B1–B6)	*Essential*—without these, metric optimization and data flow are broken.
Sqrt class weights	**Critical**—largest contributor; without it, Macro F1 $\approx$ 0.09.
Stratified subsample	**High**—reduces human share from 88% to $\sim$41%.
Label smoothing	**Moderate**—prevents overconfidence, complementary to weights.
Weight decay 0.05	**Moderate**—controls overfitting (cf. Exp. 1 with wd=0.01).
Cosine LR	**Moderate**—smooth decay suits imbalanced fine-tuning.
Max len 256	**Moderate**—+1.6–3.1% Macro F1 over len 128.
Early stopping	**Preventative**—avoids overfitting and wasted compute.

Table 12: Qualitative ablation: estimated impact of each improvement on Subtask B.

7.6 Subtask C: Hybrid Code Detection (Baseline)

For Subtask C, we have so far reproduced the official CodeBERT baseline using the provided starter notebook. The baseline fine-tunes `microsoft/codebert-base` with a 4-class classification head (human, machine, hybrid, adversarial) using standard cross-entropy loss and the same training configuration as the other subtasks.

7.6.1 Baseline Setup

The Task C baseline was executed with the following configuration:

- **Model**: `microsoft/codebert-base` with `num_labels=4`.
- **Data**: 10% subsample of the 900K training set ($\sim$90K samples) and corresponding validation subsample, to enable rapid iteration within Kaggle session limits.
- **Training**: 5 epochs, batch size 16, learning rate 2×10^{-5}, max sequence length 128.
- **Loss**: Standard cross-entropy (no class weighting or label smoothing applied yet).

7.6.2 Observations and Challenges

Unlike Subtask B where the class imbalance is extreme (88% majority class), Subtask C presents a more moderate but still meaningful imbalance: the human class comprises $\sim$54% of training data, while the hybrid class is the smallest at $\sim$9%. Key challenges identified from the baseline run include:

- **Adversarial–human confusion**: The adversarial class (code generated to mimic human style) is expected to be particularly difficult to separate from genuine human code, as it is specifically designed to evade detection.
- **Hybrid ambiguity**: Hybrid code contains contributions from both human and machine sources, making it inherently ambiguous—the transition points between human and machine segments are not annotated.
- **Same baseline bugs**: The Task C starter notebook shares the same implementation bugs identified in the Task B baseline (Section **??**), including `return_tensors='pt'` in map tokenization and redundant padding. These will be fixed before running improved experiments.

7.6.3 Planned Improvements

The improvements developed for Subtasks A and B provide a clear roadmap for Subtask C:

1. **Bug fixes**: Apply all six baseline fixes (Table **??**) to the Task C pipeline.
2. **Class-weighted loss**: Adapt the sqrt-scaled class-weighting strategy to the 4-class distribution, up-weighting the underrepresented hybrid (9%) and adversarial (13%) classes.
3. **Full-data training**: Train on the complete 900K training set rather than the 10% subsample.
4. **Sequence length**: Increase max length to 256 tokens, consistent with findings from Subtask B (Section **??**).
5. **Backbone and head experiments**: Evaluate UniXcoder and GraphCodeBERT backbones, and the classification head variants (Bi-LSTM, dense blocks) already explored for Subtask A.
6. **Ensemble**: Include Task C models in the multi-model ensemble pipeline (Phase 5).

8 Error Analysis

This section presents key findings from our Subtask B error analysis based on the per-class results in Table **??**.

8.1 LLM Family Detectability Spectrum

Our analysis shows that the 10 LLM families fall into three detectability tiers. OpenAI, IBM-Granite, Phi, and BigCode are relatively easy to detect (F1 0.48), likely because they produce code with distinctive structural patterns such as characteristic comment styles, consistent variable naming, or recognizable code organization. In contrast, DeepSeek and Mistral AI are the hardest to detect (F1 0.20; precision 0.12–0.14), suggesting their generated code closely resembles human-written code when using token-level features alone.

8.2 Precision–Recall Trade-offs

The class-weighted loss creates interesting precision–recall trade-offs:

- **Human class**: Near-perfect precision (1.00) but reduced recall (0.90)—the model successfully avoids labeling LLM code as human, but 10% of human code is misclassified as machine-generated. This is expected given our stratified subsampling which reduces human prevalence.
- **IBM-Granite**: High recall (0.73) but lower precision (0.48)—the model identifies most IBM-Granite samples correctly but also incorrectly labels some samples from other classes as IBM-Granite. This class appears to act as a partial "catch-all" for ambiguous LLM-generated code.
- **DeepSeek-AI and Mistral**: Both show low precision (0.12, 0.14) with moderate recall (0.46, 0.26)—the model attempts to identify these classes but generates many false positives, suggesting their stylistic features overlap heavily with other classes.

8.3 Overfitting and Generalization Concerns

The validation loss divergence observed in Experiment 1 (Section **??**) raises concerns about generalization to unseen authors and languages. The model performs well on the human class but poorly on minority LLM classes, suggesting it may rely on superficial frequency-based cues rather than deeper stylistic patterns. This behavior is typical in highly imbalanced datasets, where minority-class signals are underrepresented in the learned embedding space.

8.4 Planned Further Analysis

The following analyses remain for the final submission:

- **Confusion matrix**: Identifying which LLM families are most frequently confused with each other.
- **Code length effects**: Whether longer or shorter snippets systematically cause more errors.
- **Language-wise error distribution**: Performance breakdown across programming languages (C++, Python, Java).
- **Domain transfer**: Qualitative analysis of failure cases on unseen domains (research, production code).
- **Subtask C error patterns**: Analysis of the adversarial–human confusion boundary and hybrid class misclassification modes once improved Task C models are trained.

9 Progress Report

Table **??** summarizes the current status of each project component.

Objective	Status
Environment setup & data exploration	✓
Dataset download, verification, EDA	✓
Baseline (CodeBERT) reproduction, Tasks A & B	✓
Baseline (CodeBERT) reproduction, Task C	✓
Baseline bug audit and fixes (6 bugs)	✓
Kaggle Phase 1 submission	✓
Class imbalance handling (weighted loss, subsampling)	✓
UniXcoder experiment (Task B, Macro F1 = 0.470)	✓
CodeBERT HP search (Task B, Macro F1 = 0.368)	✓
Task A ablation (CodeBERT, UniXcoder, heads)	✓
CodeBERT full training with best HP (Task B, F1=0.420)	✓
Task C bug fixes and full-data baseline	Planned
Task C improved models (cls. wt., backbone exps.)	Planned
GraphCodeBERT experiments	Planned
Ensemble strategy implementation	Planned
Final evaluation on private test set	Planned
Paper finalization	Planned

Table 13: Project progress as of March 2026.

Completed work.

1. Reproduced the official CodeBERT baseline for Subtasks A, B, and C, and identified six critical implementation bugs (Table **??**).

2. Conducted exploratory data analysis, characterizing class distributions (Tables **??** and **??**), code length statistics, and language coverage.

3. Implemented and evaluated class imbalance mitigation strategies: sqrt-scaled class-weighted CrossEntropyLoss, label smoothing, and stratified subsampling.

4. Trained UniXcoder on Subtask B with class-weighted loss and stratified subsampling, achieving Macro F1 = 0.470 (Experiment 1).

5. Conducted a 3×2 hyperparameter grid search over learning rate and sequence length for CodeBERT on Subtask B, identifying the optimal configuration (LR=2×10^{-5}, max_length=256, Macro F1 = 0.368 in 2 epochs; Experiment 2).

6. Trained CodeBERT with the best hyperparameters for 5 full epochs on Subtask B (Experiment 3), achieving Macro F1 = 0.4203 with stable validation loss, confirming that the regularisation suite prevents overfitting.

7. Performed Task A ablation studies across backbone models and classification head architectures (Table in Section **??**).

8. Ran the Task C CodeBERT baseline on a 10% data subsample, establishing reference performance and identifying shared baseline bugs that need fixing before improved experiments.

Ongoing work.

1. Fine-tuning GraphCodeBERT on both Task A and Task B to evaluate the contribution of data-flow-aware pretraining.

2. Applying baseline bug fixes to the Task C pipeline and running corrected full-data baseline training.

Planned work.

1. Implement ensemble pipeline: soft voting and weighted averaging across CodeBERT, GraphCodeBERT, and UniXcoder predictions.

2. Evaluate on unseen-language and unseen-domain test splits.

3. Conduct confusion matrix analysis and per-language error breakdown.

4. Explore hierarchical classification (binary then multi-class) as an alternative to flat 11-class classification.

5. Apply class-weighted loss, label smoothing, backbone and head experiments (UniXcoder, GraphCodeBERT, Bi-LSTM, dense blocks) to Subtask C, targeting improved separation of the adversarial and hybrid classes.

6. Investigate adversarial-specific features for Task C, such as stylistic consistency metrics and perplexity-based signals, to distinguish adversarial code from genuine human code.

7. Generate final predictions for private test set across all three subtasks.

10 Timeline

Dates	Milestone
Jan 31 – Feb 07	**Setup & Baseline**: Environment setup, data exploration, CodeBERT baseline reproduction (Tasks A, B, C) and Kaggle Phase 1 submission.
Feb 08 – Feb 22	**Bug Audit & Imbalance Handling**: Baseline bug identification and fixes (6 bugs); sqrt-scaled class-weighted loss, label smoothing, stratified subsampling; UniXcoder Experiment 1 (Task B, F1=0.470).
Feb 23 – Mar 08	**HP Search & Ablation**: CodeBERT HP grid search and full 5-epoch training (Task B, F1=0.420); Task A ablation across backbones and heads; Task C baseline run.
Mar 09 – Mar 25	**Model Expansion**: GraphCodeBERT fine-tuning; Task C bug fixes, class-weighted loss, and backbone experiments; ensemble pipeline development.
Mar 26 – Apr 08	**Finalization**: Full evaluation on private test set for Tasks A, B, and C; error analysis; report writing and presentation.

Table 14: Project timeline (Spring 2026).

11 Conclusion

We have presented the approach of team Procastic Simulators to SemEval-2026 Task 13 on detecting machine-generated code. Our work addresses all three subtasks, with the primary focus to date on

Subtask B (11-class authorship attribution), where extreme class imbalance (88.4% human) poses the primary challenge, and Subtask A (binary detection), where we explore backbone and classification head alternatives.

Our key contributions are: (1) identification and correction of six critical bugs in the provided baseline that silently degraded performance; (2) a systematic approach to class imbalance through sqrt-scaled class-weighted cross-entropy loss, label smoothing ($\alpha = 0.1$), and stratified subsampling; (3) a rigorous hyperparameter search that established the optimal configuration (LR=2×10^{-5}, max_length=256); and (4) detailed per-class analysis revealing a clear hierarchy of LLM family detectability.

Our best result—Macro F1 of 0.470 with UniXcoder and class-weighted training—represents a $>2\times$ improvement over the corrected baseline ($\sim$0.22). Full training of CodeBERT with the best hyperparameters and regularisation (Experiment 3) achieves Macro F1 = 0.4203 in 5 epochs with *decreasing* validation loss, confirming that the regularisation suite prevents overfitting and suggesting further gains with extended training. Per-class analysis reveals that OpenAI (F1=0.59), IBM-Granite (0.58), Phi (0.50), and BigCode (0.48) are the most detectable LLM families, while DeepSeek-AI (0.20) and Mistral (0.19) remain the most challenging.

For Subtask A, we performed ablation studies across backbone models (CodeBERT, UniXcoder) and different classification heads (linear, Bi-LSTM, and dense layers with stylometric features). Results show that the CodeBERT baseline with a simple linear head remains competitive on the small evaluation set.

For Subtask C (4-class hybrid code detection), we reproduced the official CodeBERT baseline, establishing reference performance and identifying bugs in the baseline implementation. The techniques developed for Subtasks A and B—such as better imbalance handling, backbone exploration, and improved classification heads—provide a strong foundation for improving Task C.

Future work includes longer training, experiments with GraphCodeBERT to capture dataflow features, improved Task C models with class weighting and backbone exploration, ensemble methods across models, and broader evaluation on unseen languages and domains.